

Módulo 5: Webhooks y Consistencia de Datos

Objetivo: Escuchar las confirmaciones asíncronas de Stripe para actualizar el estado de la donación en la base de datos de manera segura.

En este módulo se completa el flujo de pago de extremo a extremo (End-to-End). Un pago simulado con tarjeta de prueba en Stripe Checkout actualizará automáticamente el registro del usuario en la base de datos SQL Server, registrando el id de la transacción y el estatus.

Confirmar un pago en Stripe y actualizar la base de datos de forma segura requiere el uso de Webhooks. A diferencia de la redirección de usuario (página de éxito), los webhooks permiten que Stripe notifique a tu servidor en tiempo real cuando un pago se completa con éxito, incluso si el usuario cierra el navegador. En modo desarrollo, utilizaremos el Stripe CLI para enviar estos eventos a la máquina local.

Un **webhook** es una función de devolución de llamada HTTP que permite a una aplicación enviar datos automáticamente a otra en tiempo real cuando ocurre un evento específico. A diferencia de una API tradicional, que requiere consultas continuas (polling), el webhook actúa como una alerta, enviando información instantáneamente solo cuando es necesario.

Características y Funcionamiento Clave:

- **Automatización:** Conectan aplicaciones entre sí sin intervención manual.
- **Basados en eventos:** Se activan ante acciones como una compra, un nuevo registro o un mensaje.
- **Eficiencia:** Ahorran recursos al eliminar la necesidad de consultar repetidamente por actualizaciones.
- **Estructura:** Se configuran proporcionando una URL única (punto final o endpoint) al servidor emisor, que recibe la información (usualmente vía HTTP POST).

Conceptos Clave

- **Webhook:** Una URL en tu servidor (ej. /webhook) que escucha eventos POST enviados por Stripe.
- **Evento payment_intent.succeeded:** El evento más común para confirmar un pago exitoso.
- **Firma del Webhook (signing secret):** Un código secreto para verificar que la solicitud realmente proviene de Stripe, esencial para la seguridad.
- **Modo Desarrollo:** Uso de sk_test_... y Stripe CLI para simular pagos.

Flujo de Trabajo

- Stripe detecta que el pago fue exitoso.
- Stripe envía un POST con datos JSON a tu URL de webhook.
- El servidor verifica la firma del evento.
- Si la firma es válida, tu servidor actualiza la base de datos (por ejemplo, cambiar estado de pendiente a pagado).
- El servidor responde con un código 200 OK a Stripe para confirmar la recepción.

Tareas clave en la implementación:

- Crear un API Controller (StripeWebhookController.cs) para recibir eventos HTTP POST desde Stripe.
- Configurar el endpoint para escuchar específicamente el evento checkout.session.completed.
- Validar la firma del Webhook usando el Webhook Secret de Stripe.
- Actualizar la base de datos: Buscar la Donation por su SessionId y cambiar el Status de "Pending" a "Completed".

Paso 1: Habilitar Controladores de API en Blazor

Los componentes Blazor Interactive Server se comunican a través de SignalR (*WebSockets*). Sin embargo, Stripe necesita enviar su notificación a un endpoint HTTP tradicional (una API REST).

Abrir y editar `Program.cs`:

```
1 using Stripe;
2 using Microsoft.AspNetCore.Components.Authorization;
3 using Microsoft.AspNetCore.Identity;
4 using Microsoft.EntityFrameworkCore;
5 using PonceDonations.Components;
6 using PonceDonations.Components.Account;
7 using PonceDonations.Data;
8 using PonceDonations.Services;
9
10 var builder = WebApplication.CreateBuilder(args);
11
12 // 1. Configuración de servicios en el contenedor de dependencias
13 builder.Services.AddRazorComponents()
14     .AddInteractiveServerComponents();
15
16 builder.Services.AddCascadingAuthenticationState();
17 builder.Services.AddScoped<IdentityRedirectManager>();
18 builder.Services.AddScoped<AuthenticationStateProvider, IdentityRevalidatingAuthenticationStateProvider>();
19 // 2. Configuración de Identity para la autenticación y autorización
20 builder.Services.AddAuthentication(options =>
21 {
22     options.DefaultScheme = IdentityConstants.ApplicationScheme;
23     options.DefaultSignInScheme = IdentityConstants.ExternalScheme;
24 })
25     .AddIdentityCookies();
26 //3. Configuración de Entity Framework Core con SQL Server
27 var connectionString = builder.Configuration.GetConnectionString("DefaultConnection") ??
28     throw new InvalidOperationException("Connection string 'DefaultConnection' not found.");
29 builder.Services.AddDbContext<ApplicationDbContext>(options =>
30     options.UseSqlServer(connectionString));
31 builder.Services.AddDatabaseDeveloperPageExceptionFilter();
32
33 builder.Services.AddIdentityCore<ApplicationUser>(options =>
34 {
35     options.SignIn.RequireConfirmedAccount = true;
36     options.Stores.SchemaVersion = IdentitySchemaVersions.Version3;
37 })
38     .AddEntityFrameworkStores<ApplicationDbContext>()
39     .AddSignInManager()
40     .AddDefaultTokenProviders();
41
42 builder.Services.AddSingleton<IEmailSender<ApplicationUser>, IdentityNoOpEmailSender>();
43
44 // A. Configurar la clave secreta de Stripe para que pueda ser utilizada en los servicios de pagos
45 StripeConfiguration.ApiKey = builder.Configuration["Stripe:SecretKey"];
46
47 // B. Registrar el servicio de StripeCheckoutService para manejar la lógica de pagos con Stripe
48 builder.Services.AddScoped<StripeCheckoutService>();
49 // C. Agregar servicios para controladores API
50 builder.Services.AddControllers();
51
52 var app = builder.Build();
53
54 // 4. Configuración del pipeline de middleware HTTP
55 if (app.Environment.IsDevelopment())
56 {
57     app.UseMigrationsEndPoint();
58 }
59 else
60 {
61     app.UseExceptionHandler("/Error", createScopeForErrors: true);
62     // The default HSTS value is 30 days. You may want to change this for production scenarios, see https://
63     app.UseHsts();
64 }
65 app.UseStatusCodePagesWithReExecute("/not-found", createScopeForStatusCodePages: true);
66 app.UseHttpsRedirection();
67 app.UseAntiforgery();
68 app.MapStaticAssets();
69
70 // Mapear controladores API para que puedan ser accedidos desde el frontend Blazor
71 app.MapControllers();
72 app.MapRazorComponents<App>()
73     .AddInteractiveServerRenderMode();
74
75 // 5. Configuración de rutas para Identity
76 app.MapAdditionalIdentityEndpoints();
77
78 app.Run();
```

Paso 2: Crear el Controlador del Webhook

- A. Crea una nueva carpeta en la raíz del proyecto llamada **Controllers**.
- B. Dentro de esta carpeta, crear el archivo **WebhookController.cs**.

```
1 using Microsoft.AspNetCore.Mvc;
2 using PonceDonations.Data;
3 using Stripe;
4 using Stripe.V2.Core;
5
6 namespace PonceDonations.Controllers;
7
8 [Route("api/[controller]")]
9 [ApiController]
10 // Este controlador maneja los webhooks de Stripe para actualizar el estado de las donaciones
11 public class WebhookController(IConfiguration config, IServiceProvider serviceProvider) : ControllerBase
12 {
13     [HttpPost]
14     public async Task<IActionResult> Index()
15     {
16         var json = await new StreamReader(HttpContext.Request.Body).ReadToEndAsync();
17         var endpointSecret = config["Stripe:WebhookSecret"];
18
19         try
20         {
21             // Verificamos que la petición realmente venga de Stripe usando la firma digital
22             var stripeEvent = EventUtility.ConstructEvent(
23                 json,
24                 Request.Headers["Stripe-Signature"],
25                 endpointSecret
26             );
27
28             // Filtramos solo el evento de pago completado
29             if (stripeEvent.Type == "checkout.session.completed")
30             {
31                 var session = stripeEvent.Data.Object as Stripe.Checkout.Session;
32                 var donationIdStr = session?.ClientReferenceId; // Aquí llega nuestro Donation.Id
33
34                 if (int.TryParse(donationIdStr, out int donationId))
35                 {
36                     // Crear un nuevo scope para obtener el DbContext (necesario en controladores/webhooks)
37                     using var scope = serviceProvider.CreateScope();
38                     var dbContext = scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();
39
40                     var donation = await dbContext.Donations.FindAsync(donationId);
41                     if (donation != null && donation.Status != "Completed")
42                     {
43                         donation.Status = "Completed";
44                         donation.StripeSessionId = session?.Id; // Guardamos el ID real de Stripe
45
46                         await dbContext.SaveChangesAsync();
47                     }
48                 }
49             }
50             // Stripe requiere un código HTTP 200 rápido para saber que recibimos el mensaje
51             return Ok();
52         }
53         catch (StripeException)
54         {
55             // Si la firma es inválida o alguien intenta falsificar la petición, retorna 400
56             return BadRequest();
57         }
58     }
59 }
```

Actualizar Repositorio en GitHub y los commits necesarios.

Paso 3: Probar el Webhook Localmente (Stripe CLI)

Como tu aplicación se está ejecutando en localhost, Stripe no puede enviar la petición por internet hasta tu computadora. Para esto, Stripe provee una herramienta oficial.

- a) Descarga e instala [Stripe CLI](#) (buscar en Google "Stripe CLI install" e instalarlo).

1 Install the Stripe CLI

From the command line, use an install script or download and extract a versioned archive file for your operating system to install the CLI.

homebrew apt yum Scoop macOS Linux **Windows** Docker

To install the Stripe CLI on Windows without Scoop:

- 1 Download the latest windows zip file from [GitHub](#).
- 2 Unzip the stripe_X.X.X_windows_x86_64.zip file.
- 3 Add the path to the unzipped stripe.exe file to your Path environment variable. To learn how to update environment variables, see the [Microsoft PowerShell documentation](#).

Windows anti-virus scanners occasionally flag the Stripe CLI as unsafe. This is very likely a false positive. For more information, see [Issue #692](#) in the GitHub repository.

- b) Abrir tu terminal PowerShell con la App Ejecutando e inicia sesión:

stripe login

- c) Ejecuta tu aplicación en Visual Studio y fíjate en qué puerto se está ejecutando (por ejemplo, https://localhost:7123).

- d) Indicar a la consola de Stripe que escuche los eventos y los reenvíe a tu controlador local:

stripe listen --forward-to https://localhost:7123/api/webhook

- e) La consola de Stripe te mostrará un mensaje clave: Ready! Your webhook signing secret is whsec_...

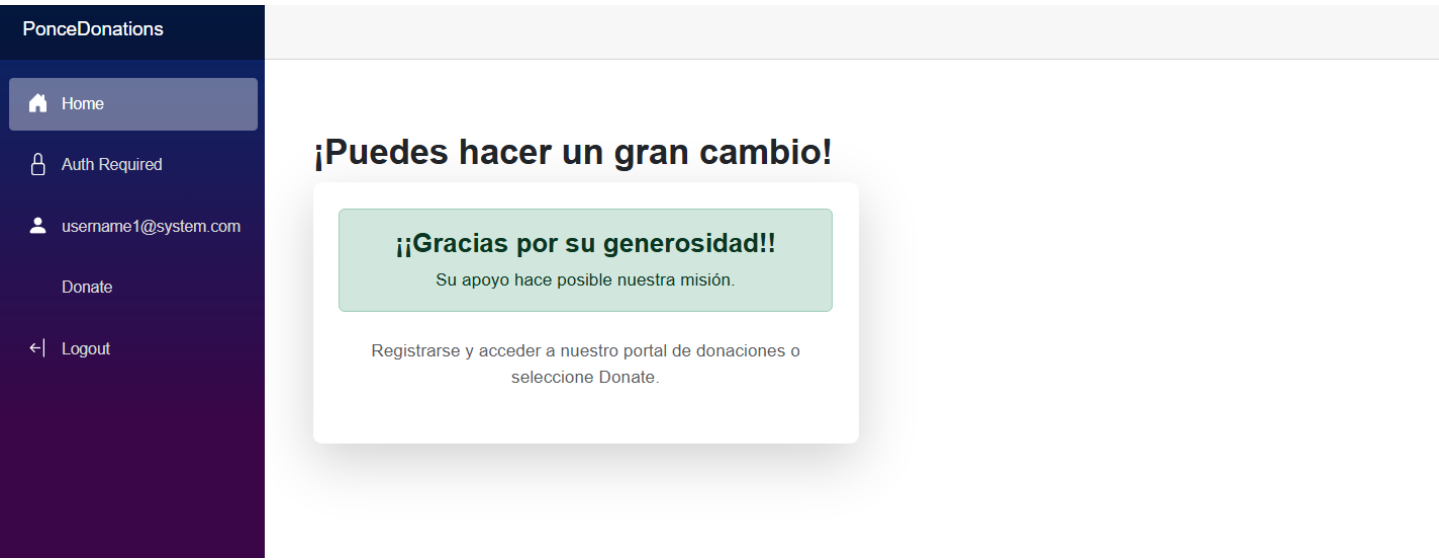
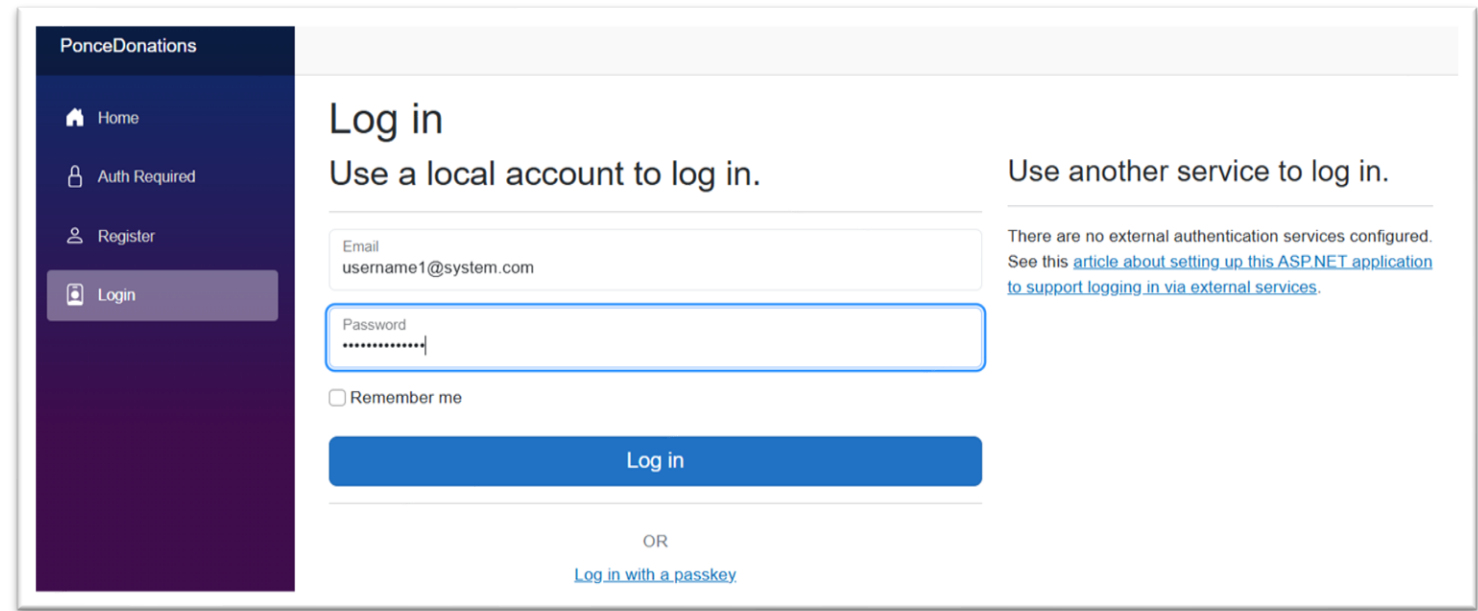
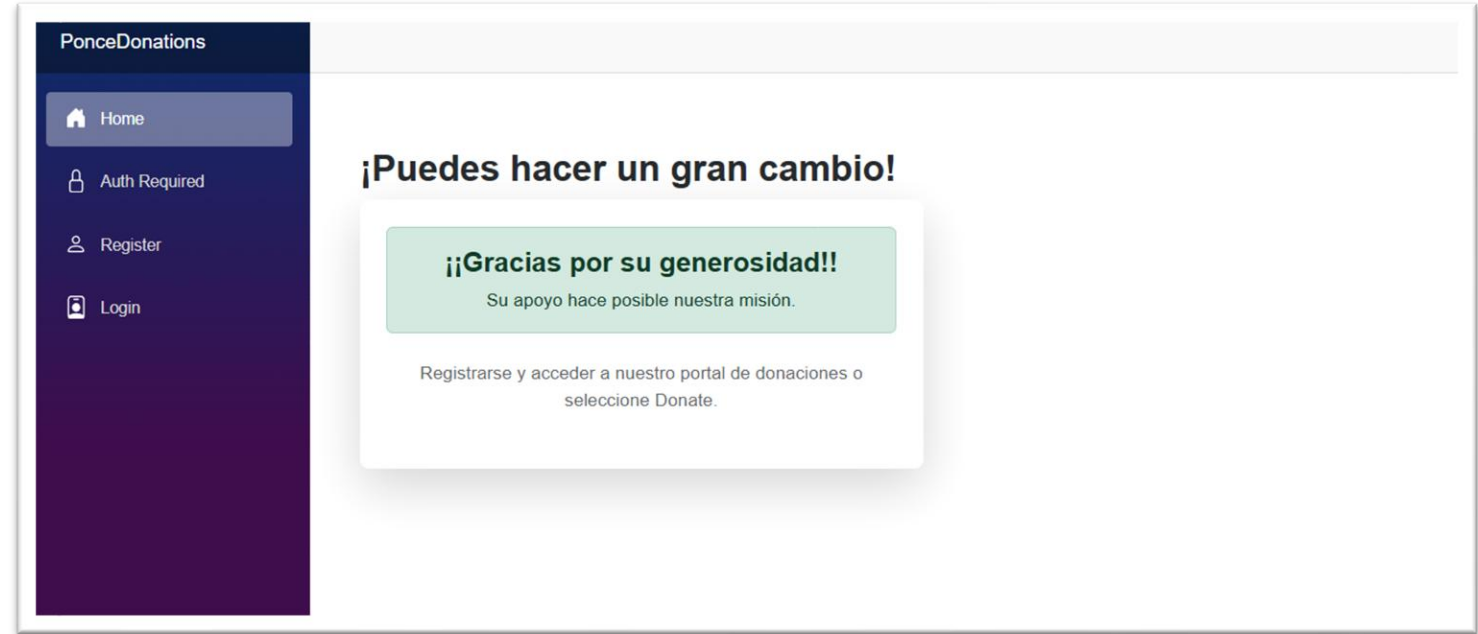
Copiar el código que empieza con whsec_.

Paso 4: Actualizar los User Secrets

Abrir nuevamente **User Secrets** en Visual Studio y actualiza el campo WebhookSecret con el valor que muestra la terminal en el paso anterior:

```
{
  "Stripe": {
    "PublishableKey": "pk_test_...",
    "SecretKey": "sk_test_...",
    "WebhookSecret": "whsec...aqui"
  }
}
```





PonceDonations

Home

Auth Required

username1@system.com

Donate

Logout

Apoya nuestra causa

¡Gracias por su donación!

Su apoyo hace posible nuestra misión.

Seleccione el fondo al que desea donar y la cantidad de la donación y presione el botón de donar.

Seleccionar Fondo

Educación

Monto a donar (USD)

\$ 22.5

Donar vía Stripe

← New business sandbox Sandbox

Donation

\$22.50

Thank you for supporting our cause!

Pay with link

OR

Contact information

Email

username1@system.com

Payment method

Card

Card information

4242 4242 4242 4242

12 / 29

CVC

VISA

Cardholder name

Juan de Ponce

Country or region

Puerto Rico

Bank

\$5 back

Klarna

Powered by Link

Save my information for faster checkout

Pay securely at New business sandbox and everywhere Link is accepted.

Pay

Powered by stripe | Terms Privacy

PonceDonations

Home

Auth Required

username1@system.com

Donate

Logout

¡Gracias por tu donación!

Tu pago se ha procesado correctamente. Apreciamos mucho tu apoyo.

Volver al inicio

dbo.Donations [Data]							
PonceDonations.csproj Donate.razor WebhookController.cs							
Max Rows: 1000							
	Id	UserId	Amount	Currency	Status	StripeSessio...	CreatedAt
▶	1	19913186-d...	12.00	usd	Pending	NULL	5/1/2026 3:3...
	2	f18f488d-99...	16.50	usd	Pending	NULL	5/1/2026 2:3...
	3	19913186-d...	20.50	usd	Pending	NULL	5/1/2026 3:4...
	4	19913186-d...	10.00	usd	Pending	NULL	5/1/2026 3:4...
	5	19913186-d...	10.00	usd	Pending	NULL	5/1/2026 3:5...
	6	19913186-d...	10.00	usd	Pending	NULL	5/1/2026 11:...
	7	19913186-d...	22.50	usd	Pending	NULL	5/2/2026 12:...
	8	19913186-d...	10.50	usd	Pending	NULL	5/2/2026 12:...
	9	f18f488d-99...	14.50	usd	Pending	NULL	5/2/2026 1:1...
	10	f18f488d-99...	16.00	usd	Completed	cs_test_a1bA...	5/2/2026 1:1...
	11	19913186-d...	7.50	usd	Completed	cs_test_a1gk...	5/2/2026 1:2...
	12	19913186-d...	13.50	usd	Pending	NULL	5/16/2026 4:...
	13	19913186-d...	7.50	usd	Completed	cs_test_a1j34...	5/16/2026 4:...
	14	f18f488d-99...	22.50	usd	Pending	NULL	5/16/2026 1:...
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Los registros correctamente procesados son las que aparecen con el estatus “Completed”